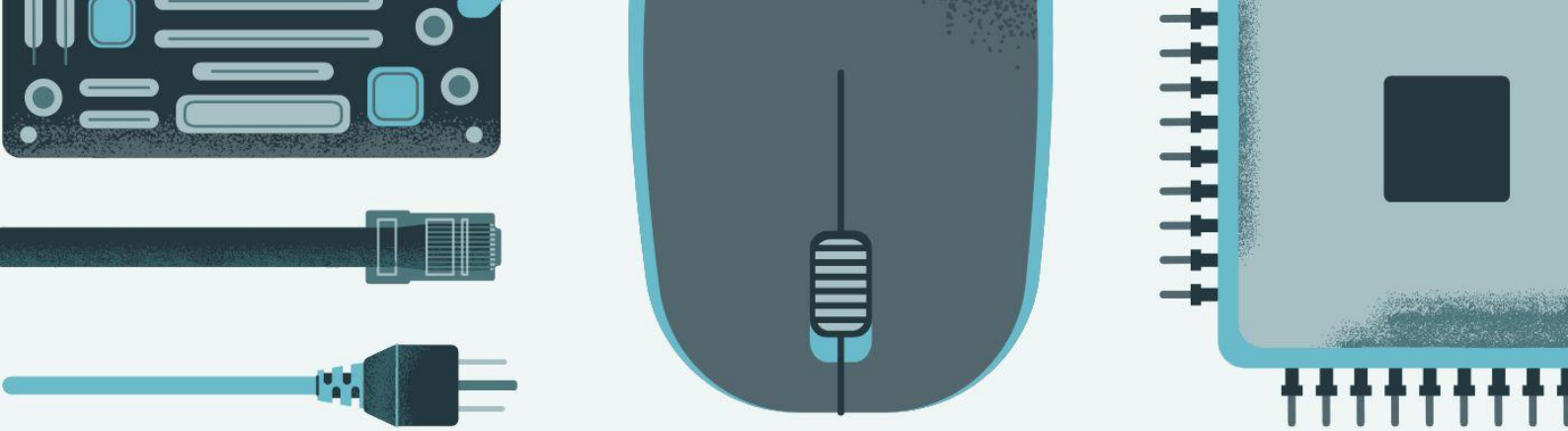




TRABAJO FINAL INTEGRADOR

LABORATORIO DE C

Carrera: Ingeniería en Sistemas de Información

Asignatura: Algoritmos y Estructuras de Datos

Año: 2025

Grupo 31

Integrantes:

Selene Magalí Llull

Guillermina Sampayo

Darío O. Rolón



Simulación de Navegación Autónoma de un Robot en un Entorno Mapeado utilizando el Algoritmo A* (A-star)

Introducción

Nuestro proyecto se desarrolló como parte del Trabajo Final Integrador de la materia Algoritmos y Estructuras de Datos, con el objetivo de aplicar los conocimientos adquiridos sobre programación estructurada, estructuras dinámicas y diseño de algoritmos.

Diseñamos una aplicación en lenguaje C, que simula la navegación autónoma de un robot dentro de un entorno bidimensional representado por una matriz. En este entorno, el robot debe desplazarse desde un punto de inicio hasta un destino, evitando obstáculos como muebles o estantes, y calculando la ruta más corta posible.

La idea surgió al investigar cómo los videojuegos y robots reales determinan el mejor camino para moverse en escenarios complejos. Durante esa búsqueda nos encontramos con el algoritmo A* (A-star), un método de búsqueda informada que combina precisión y eficiencia. A partir de ahí, decidimos implementarlo utilizando los recursos aprendidos en la materia y con apoyo de herramientas de inteligencia artificial generativa para entender mejor su funcionamiento y adaptarlo a nuestro código.

El proyecto nos permitió unir teoría, práctica e investigación, aplicando estructuras de datos vistas en clase y razonamiento algorítmico para resolver un problema real de la ingeniería: la planificación de rutas eficientes en entornos con obstáculos.

Problema que buscamos resolver – Escenario

El problema que nos propusimos resolver fue simular cómo un robot autónomo puede desplazarse dentro de un entorno con obstáculos fijos, encontrando la ruta más corta entre dos puntos sin chocar ni quedarse bloqueado.

El entorno se representa mediante una matriz bidimensional (2D), donde cada celda puede tener distintos estados:

- Libre: el robot puede moverse por allí.
- Ocupada: representa un obstáculo, como un mueble o un estante.
- Inicio o destino: marcan los puntos de partida y llegada del robot.

El usuario puede definir el tamaño de la matriz (por ejemplo, 10x10), ingresar la posición de inicio y destino, y agregar obstáculos de manera manual. El programa luego calcula la mejor ruta y la muestra en consola.

Durante la simulación, el robot es representado con “-.-”, los obstáculos con una “X” (mueble) o una “I” (estante), y las flechas indican la dirección del recorrido óptimo.

Este escenario representa situaciones reales que se dan en sistemas de logística automatizada o robótica, como en un almacén donde un robot debe moverse entre estanterías para recoger o dejar productos, sin chocar y optimizando su trayecto.

Solución propuesta

Nuestra solución consiste en un programa modular en C estándar, que implementa el algoritmo A* (A-star) para encontrar la ruta más corta dentro de la matriz, utilizando tres estructuras de datos principales: matriz, registros y lista enlazada.

La Matriz: es la estructura base del entorno. Cada posición representa una celda que puede estar libre, ocupada o designada como inicio/destino.

Los Registros (structs): los usamos para guardar información más compleja. Creamos un registro para los tipos de obstáculos (por ejemplo, “mueble” o “estante”) y otro para las celdas del mapa, que almacenan las coordenadas, los costos G, H y F, y un puntero al nodo padre, lo que permite reconstruir el camino final.

La Lista enlazada: se utiliza para manejar la llamada Lista Abierta del algoritmo A*, donde se guardan las celdas que están pendientes de ser exploradas. En cada paso, el programa busca la celda con el menor costo total $F = G + H$ y la evalúa, agregando sus vecinos a la lista si aún no fueron explorados.

El algoritmo A* funciona evaluando los posibles caminos hacia el destino. Para cada celda calcula:

- G: el costo real (cantidad de pasos desde el inicio).
- H: la estimación de la distancia restante (heurística).
- F: la suma de ambos valores ($F = G + H$).

El algoritmo siempre elige la celda con el menor valor de F, lo que le permite “pensar” cuál es el mejor camino sin explorar toda la matriz. En nuestro caso, utilizamos la distancia Manhattan como función heurística, ya que el movimiento es horizontal y vertical.

El resultado se muestra directamente en la consola: el robot esquivo los obstáculos, recorre el mapa y al final imprime el mensaje “¡Camino completo encontrado!”, indicando que logró alcanzar el destino con éxito.

Descripción de obstáculos técnicos y cómo los resolvimos

Durante el desarrollo del programa enfrentamos diferentes desafíos técnicos que fueron importantes para consolidar nuestros aprendizajes. Uno de los principales fue la gestión de memoria dinámica. Al trabajar con listas enlazadas, punteros y estructuras anidadas, fue necesario tener especial cuidado para evitar fugas de memoria. Realizamos pruebas repetidas y verificaciones paso a paso hasta garantizar la correcta liberación de los recursos. Otro desafío fue la implementación del algoritmo A*. Al principio, el recorrido del robot no se calculaba correctamente porque no se actualizaban bien los costos G y H. Revisamos la lógica, depuramos cada función y ajustamos las condiciones de actualización de los nodos, hasta lograr que el algoritmo identificara siempre el camino más corto. También trabajamos en la validación de datos ingresados por el usuario, agregando controles que impiden cargar coordenadas inválidas o colocar obstáculos sobre el inicio o el destino.

Por último, implementamos mejoras en la visualización por consola, eligiendo símbolos claros para distinguir el robot, los obstáculos y la ruta, logrando una simulación visualmente comprensible pese a no utilizar librerías gráficas.

Librerías externas utilizadas:

- `<math.h>` → para cálculos matemáticos y heurísticos.
- `<limits.h>` → para definir valores máximos y manejar el concepto de “infinito” en los costos.
- `<time.h>` → para generar variaciones aleatorias en los mapas de prueba y medir tiempos de ejecución.

Documentación y fuentes consultadas:

- Documentación oficial del lenguaje C (ISO/IEC 9899:2011).
- Apuntes de la cátedra Algoritmos y Estructuras de Datos – UTN FRRe.
- Videos explicativos en YouTube sobre el algoritmo A* y su aplicación en programación y desarrollo de juegos.
- Foros técnicos y recursos digitales (Stack Overflow, GeeksforGeeks, artículos sobre IA aplicada a juegos y robótica).

Estas fuentes fueron esenciales para comprender la implementación de las librerías, el manejo de estructuras dinámicas y la optimización del algoritmo.

Mención sobre el uso de inteligencia artificial generativa

Durante la etapa de investigación y desarrollo utilizamos herramientas de inteligencia artificial generativa como apoyo para comprender en profundidad el funcionamiento del algoritmo A*. Investigamos cómo se aplicaba en videojuegos y sistemas de navegación, y a partir de esa información, adaptamos su lógica a nuestro proyecto en C.

La IA nos ayudó principalmente en la búsqueda de información, explicación de conceptos teóricos y organización del código, pero la programación, prueba y depuración del algoritmo fueron realizadas por nosotros. De esta forma, el uso de la IA se integró como una herramienta de aprendizaje complementaria, no como sustituto del trabajo del grupo.

Conclusiones y aprendizajes del equipo

La realización de este proyecto fue una experiencia muy enriquecedora que nos permitió integrar teoría, práctica e investigación. Logramos desarrollar una aplicación funcional que demuestra cómo un robot puede planificar rutas inteligentes dentro de un entorno con obstáculos, aplicando el algoritmo A* de forma correcta y eficiente.

A nivel técnico, afianzamos nuestros conocimientos sobre estructuras de datos, algoritmos de búsqueda, manejo de memoria dinámica y validación de datos en C. A nivel grupal, fortalecimos nuestras habilidades de trabajo en equipo, distribución de tareas y comunicación.

Consideramos que el resultado final es una solución escalable, clara y eficiente, aplicable a situaciones reales como la navegación de robots en entornos industriales o logísticos. Este trabajo nos permitió comprender la importancia del razonamiento algorítmico y la correcta elección de estructuras de datos para resolver problemas del mundo real de manera ordenada y profesional.